Lab 28 — Mobile and Endpoint Hardening Baseline

> **SY0-701 Objective 4.1** — Given a scenario, apply common security techniques to computing resources.
>
> **Domain 4** — Security Operations (28%) · **Time:** ~10 min · **Course:** C628 CompTIA Security+ Exam Prep

Goal: Establish, deploy and verify an MDM-style endpoint baseline (passcode, screen lock, encryption, app allow-listing, sideload blocking, radio control, root detection) and prove it with a PASS/FAIL compliance report.

You'll build: A baseline policy file, a WPA3-Enterprise Wi-Fi profile, the enforced settings on the host, and a `baseline-check` compliance script that reports 9/9 PASS.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 16 (Linux hardening) is helpful background; tools installed in step 1.

Background

- A **secure baseline** is the documented minimum configuration for a class of device. Its lifecycle is **establish** (choose a benchmark such as CIS), **deploy** (push it with MDM, Group Policy or Ansible) and **maintain** (detect drift and re-apply).
- **Mobile device management (MDM)** such as Intune enforces passcode length, screen lock, full-device encryption, app allow/deny lists, Wi-Fi profiles and remote wipe, and marks a device non-compliant when any setting drifts.
- **Deployment models** decide how much control the company has: **BYOD** (employee owns it, so use a work profile and selective wipe), **COPE** (company owns it, personal use allowed) and **CYOD** (employee picks from an approved list, company owns it).
- **Sideloading** installs apps from outside the official store and **jailbreaking/rooting** removes the vendor's restrictions to gain root. Both bypass app vetting, so MDM detects and blocks them.
- Linux is the stand-in here: every control you apply maps directly to an MDM policy on a phone or a laptop.

Steps

1. Install the tools

You need the password-quality library and its test tool, dconf for desktop policy locks, cryptsetup for encryption, and jq to read the JSON policy.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y libpam-pwquality libpwquality-tools dconf-cli cryptsetup jq
```

2. Establish the baseline and the Wi-Fi profile

Write the baseline down first, as data. This is the "establish" phase, and the file is what an MDM would hold as its compliance policy. The Wi-Fi profile shows what an MDM pushes for WPA3-Enterprise: 802.1X with EAP-TLS against a RADIUS server, with Protected Management Frames required.

```
mkdir -p /etc/baseline
cat > /etc/baseline/endpoint-baseline.json <<'EOF'
{
  "policy": "C628-ENDPOINT-BASELINE v1.0",
  "passcode":    {"min_length": 12, "min_classes": 3, "max_age_days": 90},
  "screen_lock": {"idle_seconds": 300, "user_can_change": false},
  "encryption":  {"required": true, "examples": "LUKS2, BitLocker, FileVault, Android/iOS
device encryption"},
  "apps":        {"deny_list": ["telnet", "inetutils-telnet", "rsh-client"],
"sideloading": "blocked"},
  "radios":      {"bluetooth": "disabled unless an exception is approved", "wifi": "WPA3-
Enterprise only"},
  "integrity":   {"root_or_jailbreak_detection": true, "uid0_accounts_allowed": 1}
}
EOF
cat > /etc/baseline/corp-wifi-wpa3.conf <<'EOF'
# Wi-Fi profile an MDM would push (wpa_supplicant syntax). WPA3-Enterprise, EAP-TLS, PMF
required.
# For WPA3-Personal you would use key_mgmt=SAE with a passphrase instead of 802.1X.
network={
    ssid="CORP-SECURE"
    key_mgmt=WPA-EAP-SHA256
    ieee80211w=2
    eap=TLS
    identity="alice@corp.example"
    ca_cert="/etc/baseline/radius-ca.pem"
    client_cert="/etc/baseline/alice.pem"
    private_key="/etc/baseline/alice.key"
}
EOF
jq . /etc/baseline/endpoint-baseline.json
```

3. Deploy the passcode policy

Enforce length and complexity through PAM (pwquality) and password ageing through `login.defs`. Then test the policy with `pwscore`, which rejects weak passwords exactly as the login stack will.

```
mkdir -p /etc/security/pwquality.conf.d
cat > /etc/security/pwquality.conf.d/50-baseline.conf <<'EOF'
minlen = 12
minclass = 3
dictcheck = 1
retry = 3
EOF
sed -i 's/^PASS_MAX_DAYS.*/PASS_MAX_DAYS   90/; s/^PASS_MIN_DAYS.*/PASS_MIN_DAYS   1/'
/etc/login.defs
grep -E '^PASS_(MAX|MIN)_DAYS' /etc/login.defs
echo 'Summer24' | pwscore || echo "-> rejected by the baseline"
echo 'Cobalt-Harbour-Lantern-81' | pwscore
```

```
Password quality check failed:
 The password is shorter than 12 characters
-> rejected by the baseline
```

4. Enforce screen lock and idle timeout

Shells get an idle timeout. Desktops get a GNOME screen-lock policy that is **locked**, so the user cannot change it, which is the same idea as an MDM configuration profile.

```
cat > /etc/profile.d/99-idle-lock.sh <<'EOF'
# Baseline: terminate idle interactive shells after 600 seconds
TMOUT=600
readonly TMOUT
export TMOUT
EOF
mkdir -p /etc/dconf/profile /etc/dconf/db/local.d/locks
printf 'user-db:user\nsystem-db:local\n' > /etc/dconf/profile/user
cat > /etc/dconf/db/local.d/00-screenlock <<'EOF'
[org/gnome/desktop/session]
idle-delay=uint32 300

[org/gnome/desktop/screensaver]
lock-enabled=true
lock-delay=uint32 0
EOF
cat > /etc/dconf/db/local.d/locks/screenlock <<'EOF'
/org/gnome/desktop/session/idle-delay
/org/gnome/desktop/screensaver/lock-enabled
/org/gnome/desktop/screensaver/lock-delay
EOF
dconf update && ls -l /etc/dconf/db/local
```

5. Require encryption for the secure folder

Phones encrypt by default and MDM verifies it. Here you create an encrypted container, the equivalent of a work-profile or "secure folder", and confirm that it is LUKS2.

```
truncate -s 32M /root/secure-folder.img
head -c 64 /dev/urandom > /root/secure-folder.key && chmod 600 /root/secure-folder.key
cryptsetup luksFormat --batch-mode --type luks2 /root/secure-folder.img /root/secure-
folder.key
cryptsetup isLuks /root/secure-folder.img && echo "secure-folder.img is LUKS-encrypted"
cryptsetup luksDump /root/secure-folder.img | grep -iE '^version|cipher:|pbkdf:' | head -4
```

6. Enforce the app deny-list and block sideloading

An APT pin with priority -1 makes a banned package impossible to install from the repositories, which acts as a deny-list. A `noexec` mount stops apps "sideloaded" into a downloads area from running.

```
cat > /etc/apt/preferences.d/99-baseline-deny-list <<'EOF'
Package: telnet inetutils-telnet rsh-client
Pin: release *
Pin-Priority: -1
```

```
EOF
apt-cache policy telnet | head -3
mkdir -p /mnt/downloads
mountpoint -q /mnt/downloads || mount -t tmpfs -o size=16m,noexec,nosuid,nodev tmpfs
/mnt/downloads
printf '#!/bin/sh\necho "sideloaded app is running"\n' > /mnt/downloads/freegame.sh
chmod +x /mnt/downloads/freegame.sh
/mnt/downloads/freegame.sh || echo "BLOCKED: the untrusted location is mounted noexec"
```

```
telnet:
  Installed: (none)
  Candidate: (none)
bash: /mnt/downloads/freegame.sh: Permission denied
BLOCKED: the untrusted location is mounted noexec
```

Note that `sh /mnt/downloads/freegame.sh` would still run, because the shell reads the file rather than executing it. Real application allow-listing (AppArmor, fapolicyd, Windows WDAC or the MDM app catalogue) closes that gap.

7. Disable unneeded radios and services, and detect root-level tampering

Disable Bluetooth at the kernel-module level, review what is listening, then record the root-equivalent footprint (UID 0 accounts and SUID binaries). This is the Linux version of MDM jailbreak/root detection.

```
cat > /etc/modprobe.d/99-baseline-bluetooth.conf <<'EOF'
# Baseline: Bluetooth disabled unless a documented exception exists
install bluetooth /bin/false
install btusb /bin/false
EOF
systemctl disable --now bluetooth.service 2>/dev/null || echo "no bluetooth service on
this host"
ss -tulpn | awk 'NR==1 || /LISTEN/'
awk -F: '$3==0 {print "UID 0 account:", $1}' /etc/passwd
find / -xdev -perm -4000 -type f 2>/dev/null | sort > /etc/baseline/suid-baseline.txt
wc -l /etc/baseline/suid-baseline.txt
```

8. Verify compliance with a check script

This is the "maintain" phase. The script re-checks every policy item and returns non-zero on drift, which is how an MDM would flag the device as non-compliant and block access to corporate resources.

```
cat > /usr/local/sbin/baseline-check <<'EOF'
#!/bin/bash
# MDM-style compliance check against /etc/baseline/endpoint-baseline.json
pass=0; fail=0
check() { if eval "$2" >/dev/null 2>&1; then echo "PASS  $1"; pass=$((pass+1)); else echo
"FAIL  $1"; fail=$((fail+1)); fi; }
check "Passcode minimum length >= 12" "grep -Eq '^minlen *= *(1[2-9]|[2-9][0-9])'
/etc/security/pwquality.conf.d/50-baseline.conf"
check "Password maximum age <= 90 days" "awk '/^PASS_MAX_DAYS/ {exit !(\$2<=90)}'
/etc/login.defs"
check "Idle shell timeout configured" "grep -q '^TMOUT=' /etc/profile.d/99-idle-lock.sh"
check "Desktop screen lock locked" "test -s /etc/dconf/db/local.d/locks/screenlock"
check "Secure folder encrypted" "cryptsetup isLuks /root/secure-folder.img"
```

```
check "Telnet blocked by deny-list" "apt-cache policy telnet | grep -q 'Candidate:
(none)'"
check "Sideload path mounted noexec" "findmnt -no OPTIONS /mnt/downloads | grep -q noexec"
check "Bluetooth module disabled" "grep -q '^install bluetooth /bin/false'
/etc/modprobe.d/99-baseline-bluetooth.conf"
check "Only root has UID 0" "test \$(awk -F: '\$3==0' /etc/passwd | wc -l) -eq 1"
echo "Result: $pass passed, $fail failed"
[ "$fail" -eq 0 ]
EOF
chmod 755 /usr/local/sbin/baseline-check
baseline-check | tee /etc/baseline/compliance-report.txt
```

Test it

Run the compliance check. Then introduce drift by removing the `noexec` protection and run it again.

```
baseline-check; echo "exit=$?"
umount /mnt/downloads && baseline-check; echo "exit=$?"
mount -t tmpfs -o size=16m,noexec,nosuid,nodev tmpfs /mnt/downloads && baseline-check |
tail -1
```

Expected: the first run ends with `Result: 9 passed, 0 failed` and `exit=0`. After the unmount you see `FAIL Sideload path mounted noexec` with `exit=1`. Re-mounting restores 9/9.

Troubleshooting

Symptom	Fix
New terminals log out after 10 minutes idle	That is the TMOUT baseline working. For the lab, open a new shell; to remove it, rename `/etc/profile.d/99-idle-lock.sh` to `.disabled`.
`pwscore: command not found`	Install `libpwquality-tools` (step 1).
`Telnet blocked by deny-list` fails	Check the pin file name and syntax, then run `apt-cache policy telnet`; the Candidate must read `(none)`.

Challenge

Add a tenth check that fails if any SUID binary appears that is not in `/etc/baseline/suid-baseline.txt`. Prove it by running `cp /usr/bin/id /usr/local/bin/id2 && chmod 4755 /usr/local/bin/id2`, then revert with `chmod 755`.

Reflection

A sales director wants to read work email on a personal phone. Which deployment model applies, and which baseline items would you enforce through a work profile rather than on the whole device?

Exam tips

- BYOD = user owns it (privacy concerns, selective wipe); COPE = company owns it, personal use allowed; CYOD = user picks from an approved list.

- Sideloading, jailbreaking (iOS) and rooting (Android) all bypass store vetting. MDM detects them and blocks the device.
- WPA3 replaces the WPA2 pre-shared key with SAE (Personal) and adds a 192-bit suite (Enterprise). AAA for Wi-Fi is 802.1X plus RADIUS.
- Baseline lifecycle order is establish, deploy, maintain. Drift detection belongs to "maintain".

References

- CIS Benchmarks (Ubuntu Linux, Apple iOS, Google Android) — https://www.cisecurity.org/cis-benchmarks
- NIST SP 800-124 Rev. 2, Guidelines for Managing the Security of Mobile Devices — https://csrc.nist.gov/pubs/sp/800/124/r2/final
- Microsoft Intune device compliance policies — https://learn.microsoft.com/mem/intune/protect/device-compliance-get-started
- Wi-Fi Alliance WPA3 overview — https://www.wi-fi.org/discover-wi-fi/security
- MITRE ATT&CK Mobile — https://attack.mitre.org/matrices/mobile/
